

Esempio:

$X = \langle B, D, C, D \rangle$

$Y = \langle A, B, C, B, D \rangle$

esempio $LCS(X, Y)$:

		A	B	C	B	D	← Y
		0	1	2	3	4	5
B	0	0	0	0	0	0	0
	1	0	0/↑	1/↖	1/←	1/↖	1/←
D	2	0	0/↑	1/↑	1/↑	1/↑	2/↖
C	3	0	0/↑	1/↑	2/↖	2/←	2/↑
D	4	0	0/↑	1/↑	2/↑	2/↑	3/↖

↑
X

↖ B C D ← la LCS

L/B

seleziona il carattere solo quando nel cammino da $B(m, n)$ trova '↖'

Algorithm per stampare la LCS:
segue il cammino da $B(m, n)$

```
PRINT-LCS (X, B, i, j)
  if  $i=0$  or  $j=0$  then return  $\epsilon$ 
  if  $B[i, j] = \text{'\(\diagup\')}$  then
    PRINT-LCS (X, B,  $i-1, j-1$ )
    print ( $x_i$ )
  else if  $B[i, j] = \text{'\(\leftarrow\')}$  then
    PRINT-LCS (X, B,  $i, j-1$ )
  else PRINT-LCS (X, B,  $i-1, j$ )
  return
```

Complessità: $O(m+n)$

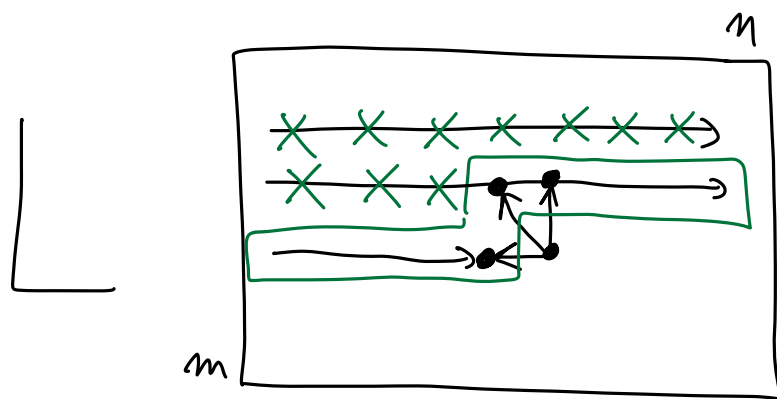
perché ad ogni chiamata ricorsiva la somma
dei 2 indici $i+j$ diminuisce di almeno 1

Complessità spaziale

= quantità di spazio di memoria richiesta da un algoritmo, in funzione della taglia dell'input

Per il calcolo di $L(m, n) : \Theta(m \cdot n)$

Si può risparmiare spazio? Sì



$$1 \text{ array} + \Theta(1) : \Theta(n)$$

se $m \ll n$ conviene una scansione column-major

$$\rightarrow \Theta(\min\{m, n\})$$

Algoritmo memorizzato

(vedremo che in alcuni casi è più efficiente)

INIT-LCS (X, Y)

(senza l'informazione
addizionale)

$m = \text{length}(X)$

$n = \text{length}(Y)$

if $m=0$ & $n=0$ then return 0

for $i=0$ to m do

$L[i, 0] = 0$

for $j=1$ to n do

$L[0, j] = 0$

for $i=1$ to m do

for $j=1$ to n do

$L[i, j] = -1$ // valore di default

return MEMO-LCS (X, Y, m, n)

$\text{PEPO-LCS}(X, Y, i, j)$

if $L[i, j] = -1$ then

if $x_i = y_j$ then

$$L[i, j] = \text{PEPO-LCS}(X, Y, i-1, j-1) + 1$$

else if $\text{PEPO-LCS}(X, Y, i-1, j) > \text{PEPO-LCS}(X, Y, i, j-1)$ then

$$L[i, j] = L[i-1, j]$$

else

$$L[i, j] = L[i, j-1]$$

return $L[i, j]$

Complexità: $O(m \cdot n)$

1 nodo interno $\forall$ sottoproblema, che sono $m \cdot n$

è al caso pessimo; ma nel caso migliore questo algoritmo ha complessità più bassa

Osservazioni: - alg. iterativa fa sempre m.n confronti
 - nell' alg. memorizzata, se $x_i = y_j$, non devo risolvere i sottoproblemi $(i-1, j)$ e $(i, j-1)$ ($\Rightarrow$ nelle rispettive posizioni in L rimane il valore di default)

Esempio:

$$X = \langle A, A, B, B, C \rangle$$

$$Y = \langle B, B, C \rangle$$

sottoistanze generate:

$$(5, 3) \xrightarrow{C=C} (4, 2) \xrightarrow{B=B} (3, 1) \xrightarrow{B=B} (2, 0)$$

$$\left\{ \begin{array}{l} \cancel{(4, 3)} \\ \cancel{(5, 2)} \end{array} \right.$$

$$\left\{ \begin{array}{l} \cancel{(4, 1)} \\ \cancel{(3, 2)} \end{array} \right.$$

$$\left\{ \begin{array}{l} \cancel{(2, 1)} \\ (3, 0) \end{array} \right.$$

$\hookrightarrow$ sottoistanze non generate

	0	1	2	3
0	0	0	0	0
1	0			
2	0		-1	
3	0	.		
4	0		.	
5	0	-1		.

risparmia rispetto all'approccio iterativo
quando le 2 stringhe sono molto simili
(cosa che accade spesso in genomica)

Caso $Y =$ suffisso di $n \leq m$ caratteri di X :

$$T_{\text{best}}(m, n) = n \quad \text{addirittura sublineare!}$$

Si può fare meglio? Bah!

$$\Omega(m+n) \quad \text{e} \quad O(m+n)$$

P.P. '80 : $O(n^2 / \log n)$ $O(n^{1.99})$?
"sembra" di no